

Coursework 2: Audio Identification

Gewenxin Yu

Queen Mary University of London

Email: `ec22214@qmul.ac.uk`

1. Introduction

The increasing popularity of audio streaming services has created a need for efficient and accurate audio search algorithms. Audio search or audio identification algorithms are essential for quickly identifying and retrieving specific pieces of audio content from large databases.

The term *fingerprint* can refer to the fingerprint of any multimedia object. As introduced by Haitsma [2], multimedia fingerprinting is a process that creates small fingerprints to efficiently determine the perceptual similarity of two multimedia objects. Rather than directly comparing large multimedia files, multimedia fingerprints are compared to determine perceptual similarity. In addition, the fingerprints always act as an index to the metadata. Fingerprinting is sometimes also referred to as robust or perceptual hashing[1].

In this paper, we will focus only on the concept of fingerprinting in the audio domain (and for audio recognition) and the implementation of the industrial strength audio search algorithm introduced by Wang [4].

2. Background

Over the past few years, several audio search algorithms have been developed and deployed commercially.

A novel audio identification engine has been developed and commercially implemented by Wang [4]. As they introduced, the designed algorithm is resistant to noise and distortion, computationally efficient, and has a high degree of scalability. The algorithm can quickly detect a short segment of music captured by a mobile phone microphone, even in the presence of foreground noise or other types of interference. In addition, the design of the algorithm is known for its robustness, which means that the generated hashes can be reproduced from a degraded copy of the original audio track.

3. Description of Algorithms

3.1. General Framework

As Haitsma [2] concludes, a fingerprint system for audio identification task typically consists of two components: a method for extracting fingerprints and a method for efficiently retrieving for the matched fingerprints in a database. The basic structure of the robust algorithm developed by Wang [4] is also consists of these two parts. Therefore, for the implementation, two functions called `fingerprintBuilder` and `audioIdentification` are mainly responsible for the whole fingerprint identification system.

3.2. Audio Fingerprints Extraction Based on Spectral Peaks

3.2.1. Design of Audio Fingerprints

In the first step, audio fingerprints are created based on the strategy of spectral peaking. This method is chosen to ensure that the fingerprints have sufficient temporal locality, translation invariance, robustness and entropy.

Given a sampled audio signal, the first step is computing the discrete STFT $\mathcal{X}$ which described by Müller(2.148) [3] :

$$\mathcal{X}(m, k) = \tilde{x}(m, k/N) = \sum_{n=0}^{N-1} x(n + mH) \tilde{w}(n) \exp(-2\pi i k n/N) \quad (1)$$

where $\mathcal{X}(n, k)$ denotes the k^{th} Fourier coefficient for the n^{th} time frame and $k \in [0: K]$, $n \in \mathbb{Z}$.

Therefore, two components including time stamp n and frequency stamp k are obtained which would later used for generating the time-frequency point $(n, k) \in \mathbb{Z} \times [0: K]$.

In the second step, the STFT $\mathcal{X}$ is firstly reduced to a sparse set of coordinates. A peak-picking strategy is then used to identify time-frequency points that have a higher magnitude than all their neighbours within a region around the respective points. Here we need to use two parameters, τ and κ to determine the size of this region. Moreover, the length on the horizontal time axis of this region is 2τ , and the height on the vertical frequency axis is 2κ . Finally, as Müller [3] described, we select the point as the peak if

$$|X(n_0, k_0)| > |X(n, k)| \quad (2)$$

for all $(n, k) \in ([n_0 - \tau, n_0 + \tau] \times [k_0 - \kappa, k_0 + \kappa]) \cap (\mathbb{Z} \times [0: K])$.

The choice of values for these two parameters (τ and κ) is also crucial. We can conclude that if this neighbourhood is too large, the number of final points will become very small, i.e. these two parameters actually determine the density of the selected peaks and to obtain a reasonably uniform coverage of the time-frequency plane.

The implemented spectral peak-picking method here extracts one-hot encoded binary array that identifies the peaks in a spectrogram. It defines a *neighborhood region* around each point, then selects the point with the highest magnitude within that region as the peak.

When the peak-selection step is complete, the original spectrogram would then be transformed into the so-called *sparse set*, which is also called the *constellation map*. The magnitude information is already eliminated here, as we only have two features of the peaks, time and frequency stamps.

The following four figures show the spectrogram and the corresponding constellation map of `classical.00000.wav` and `classical.00001.wav` in the dataset.

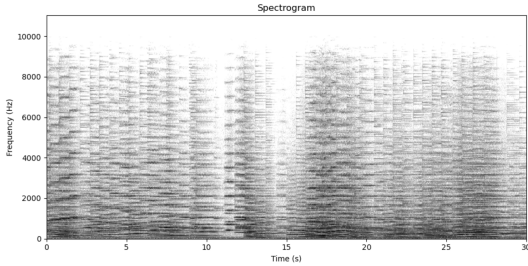


Figure 1. The STFT spectrogram of file `classical.00000.wav`

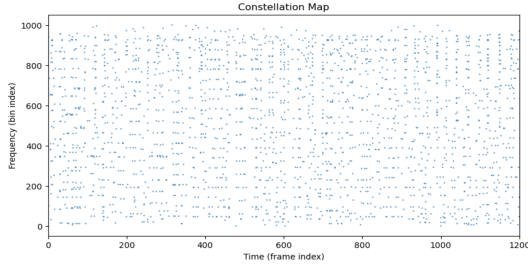


Figure 2. The constellation map of file `classical.00000.wav`

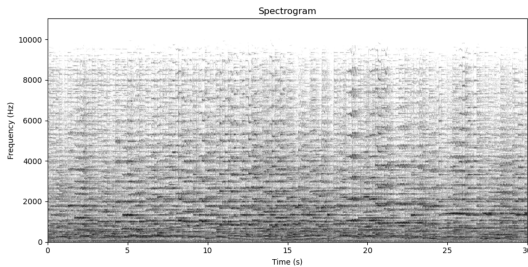


Figure 3. The STFT spectrogram of file `classical.00001.wav`

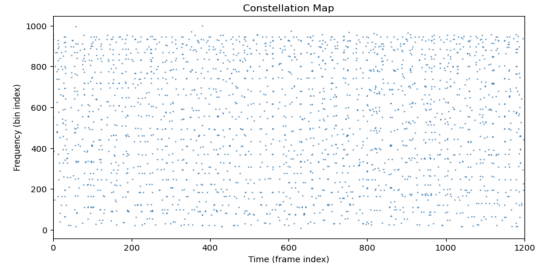


Figure 4. The constellation map of file `classical.00001.wav`

3.2.2. Fingerprint Matching

In general, the goal of an audio identification or audio fingerprinting task is to identify the original audio recording when given a short query audio fragment. Basically, all we have to do is to imagine this query audio fragment Q as a short transparent paper tape, and the whole database recording D as a much longer paper tape on which constellation maps $C(Q)$ and $C(D)$ are stored. The matching process is to move the short constellation map $C(Q)$ from front to back on the long constellation map $C(D)$, and the more overlapping points, the more likely it is that the query audio fragment is contained in the document. Here we only need to consider the significant level statistically, so even if there is noise or distortion, it doesn't really affect the fingerprint recognition.

3.3. Indexing Technique and Index-based Audio Identification

3.3.1. Indexing Technique

In an information retrieval task, indexing refers to the process of creating an organised and searchable catalogue or database of documents, data or other information resources. Hashes are always used as a mathematical function that takes in data and outputs a fixed-length string. In this case, each hash can be packed into a 32-bit unsigned integer.

With the hash created, querying songs now only requires searching the database for songs with the same hash value, rather than searching all songs, which improves efficiency, saves resources, and satisfies the requirement for the audio fingerprint to have sufficient entropy.

This is the basic idea of querying with indexed hashes. However, later on we will organise the hash in a different way, using a valid triplet pattern as a prototype before generating the hash.

Before we perform the peak-pairing process, a function called `hashBuilder` is constructed to build the hashes using `sha256`. To create a database index, the above operation is performed on each track in a database to create a corresponding list of hashes and their associated offset times.

3.3.2. Index-based Audio Identification

In response to the music industry’s demand for scalability, Wang [4] used a special method to increase the number of specificities and frequency stamps without sacrificing the robustness of the fingerprint by considering pairs of peaks rather than individual peaks, i.e. the *peak-pairing* strategy.

A fixed point which called a *anchor point* $(n_0, k_0) \in \mathcal{C}(\mathcal{D})$ is set and bound to a *target zone* $T_{(n_0, k_0)} \subset \mathbb{Z} \times [0: K]$.

This anchor point (n_0, k_0) and each time-frequency point (n_1, k_1) in its corresponding rectangular target area form a pair $((n_0, k_0), (n_1, k_1))$, and each pair of such two points yields a triple $(k_0, k_1, n_1 - n_0)$ which consists of two stamps of frequency and the time difference.

Feature representations for the database document $\mathcal{F}(\mathcal{D})$ and the query audio fragement $\mathcal{F}(\mathcal{Q})$ will be redefined using these newly designed triples:

$$\mathcal{T}(n_0, k_0) := \{(k_0, k_1, n_1 - n_0) | (n_1, k_1) \in T_{(n_0, k_0)} \cap \mathcal{C}(\mathcal{D})\} \quad (3)$$

The set $\mathcal{T}(n_0, k_0)$ can be formed by using one anchor point $(n_0, k_0) \in \mathcal{C}(\mathcal{D})$. As we can see, this gives us two frequency components and one component representing the time difference.

Additionally, the number of constellation points per target zone is called the *fan-out* factor, $F \in \mathbb{N}$.

For this purpose, the implemented `find_peak_pairs` function is used to complete the entire peak-pairing method. The input is the output of `extract_peak_fingerprints` and the arguments for defining the *fan-out* factor F of the target zone. It returns a dictionary containing the information of the pair triple and the corresponding time offset.

A function called `hashMatching` is then used for searching the database. The inputs are the hashes of the query set and the hashes of the original database document, and it returns all matching hashes. For each matching hash, the offset time from the beginning of the query sample and the database files are associated in time pairs.

After this a function called `hashSorting` is then used to build a histogram and perform the final sorting task. The significant peak of a histogram here means a match. Furthermore, if there is a significant peak in the histogram, there must also be a linear fit in the scatter plot.

3.4. Evaluation

The evaluation results showed a particularly low accuracy, perhaps due to a problem with the hash builder, or perhaps a problem with the parameter tuning. Perhaps it would be better to use a neural network approach where the parameters can be automatically optimised, as training and manually debugging the parameters each time makes the overall speed slow.

3.5. Conclusion

The good part about this implementation is that I tried to follow the Shazam paper as closely as possible, using the `sha256` encoding to construct the hash, although the results show that it seems to be difficult to match the queried music fragments with the database fingerprint. In addition, the method of using a *neighbourhood region* and the *peak-pairing* strategy is also used in this implementation.

The weakness is obvious: as the accuracy rate is not yet up to standard, the comparison of parameters and the corresponding performance is missing in this coursework.

Bibliography

- [1] J.A. Haitsma, A.A.C.M. Kalker, and J. Oostveen. Robust audio hashing for content identification. In *International Workshop on Content-Based Multimedia Indexing (CBMI'01), Brescia, Italy, September 19-21, 2001*. 2001.
- [2] Jaap Haitsma and Ton Kalker. A highly robust audio fingerprinting system with an efficient search strategy. *Journal of New Music Research*, 32:211–221, 2003.
- [3] Meinard Müller. Fundamentals of music processing: audio, analysis, algorithms, applications. 2015.
- [4] Avery Wang. An industrial strength audio search algorithm. In *International Society for Music Information Retrieval Conference*. 2003.